Howto Recipes MPI Python

MPI for Python

MPI for Python (`mpi4py`) provides Python bindings for the Message Passing Interface (MPI) standard, allowing Python applications to exploit multiple processors on workstations, clusters and supercomputers.

You can learn about `mpi4py` here: https://mpi4py.readthedocs.io/en/stable/.

Installation

If you are using a Rocky 8 node (orcd-login), then `mpi4py` is already installed in the `miniforge/24.3.0-0` module.

If you are using CentOS 7 (such as orcd-vlogin001 or orcd-vlogin002), then you need to create a new Conda environment using the `miniforge` module:

```
module load miniforge
```

And install `mpi4py` :

```
conda create -n mpi mpi4py
```

Example codes

Prepare your Python codes.

Example 1: The following code is for sending and receiving a dictionary. Save it in a file named `p2p-send-recv.py` :

p2p-send-recv.py

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
```

```
if rank == 0:
    data = {'a': 7, 'b': 3.14}
    comm.send(data, dest=1, tag=11)
    print(rank,data)
elif rank == 1:
    data = comm.recv(source=0, tag=11)
    print(rank,data)
```

Example 2: The following code is for sending and receiving an array. Save it in a file named `p2p-array.py`:

p2p-array.py

```
from mpi4py import MPI
import numpy

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

# passing MPI datatypes explicitly
if rank == 0:
    data = numpy.arange(1000, dtype='i')
    comm.Send([data, MPI.INT], dest=1, tag=77)
    print(rank,data)
elif rank == 1:
    data = numpy.empty(1000, dtype='i')
    comm.Recv([data, MPI.INT], source=0, tag=77)
    print(rank,data)

# automatic MPI datatype discovery
if rank == 0:
    data = numpy.arange(100, dtype=numpy.float64)
    comm.Send(data, dest=1, tag=13)
    print(rank,data)
elif rank == 1:
    data = numpy.empty(100, dtype=numpy.float64)
    comm.Recv(data, source=0, tag=13)
    print(rank,data)
```

Submitting jobs

Prepare a job script. The following is a job script for running `mpi4py` codes on 8 CPU cores of one node. Save it in a file named `p2p-job.sh`:

p2p-job.sh

```
#!/bin/bash -l
#SBATCH -N 1
```

```
#SBATCH -n 8
#SBATCH -p mit_normal

module load miniforge
module load openmpi/4.1.4

mpirun -np $SLURM_NTASKS python p2p-send-recv.py

mpirun -np $SLURM_NTASKS python p2p-array.py
```

Finally, submit the job:

```
sbatch p2p-job.sh
```

February 11, 2026